

Tipos, métodos e interfaces

Por: Ricardo Cuellar



Tipos, métodos e interfaces

En Go no hay clases como en otros lenguajes, pero si hay herramientas para modelar cosas del mundo real: ***tipos, métodos e interfaces.***



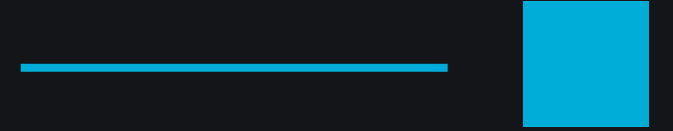
Objetivo

Al final como Golang Developer debes:

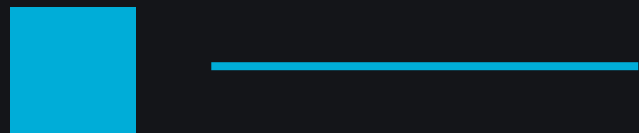
- Crear tipos propios.
- Definir métodos sobre esos tipos
- Diseñar interfaces pequeñas para desacoplar código
- Elegir entre receiver por valor vs por puntero.
- Evitar errores típicos (nil, pointers, nil trap interface)



{dev/talles}



Tipos en Go



¿Qué es un tipo en Go?

Un tipo define: que valores puedo tener, que operaciones son válidas y cómo se interpreta en memoria.

Básicos : int, string, bool, etc.

Compuestos : struct, slice, map, etc.

Definidos por nosotros: Money, Tax, Fns



¿Para qué sirven los tipos propios?

- Dar significado: Money, User, Email
- Evitar mezclar cosas por error.
- Centralizar reglas (validación, formato, helpers).
- Mejorar la legibilidad del dominio.

Ejemplo:

int64: número

Money: Es “dinero”, con reglas



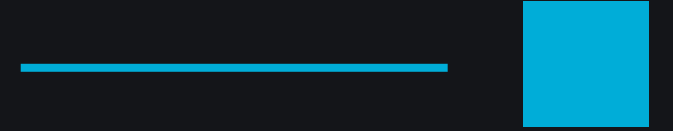
Tipos alias vs tipos definidos

`type ID = string` → alias (exactamente el mismo tipo)

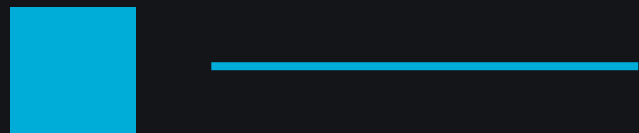
`type ID string` → tipo nuevo (diferente a string)

Si quieres seguridad y que Go nos proteja de mezclar tipos hay que usar un tipo nuevo.





Métodos en Go



¿Qué es un método en Go?

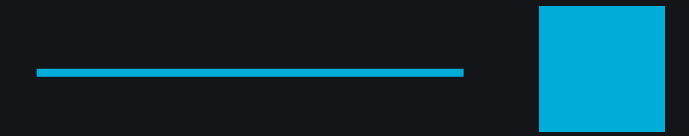
Un método es una función pegada a un tipo.

```
func (u User) FullName() string { ... }
```

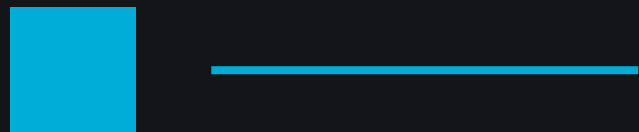
No existen las clases pero Tipo + métodos = “objeto” (datos + comportamiento).



¿Para qué sirven los métodos?



- Encapsular comportamiento del tipo
- Mantener reglas cerca de los datos
- Mejorar API del dominio
- Mejor organización.
- Más expresivo que funciones sueltas.



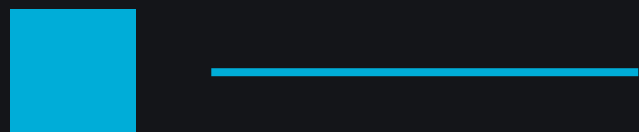
Receiver por valor vs por puntero

Receiver por valor:

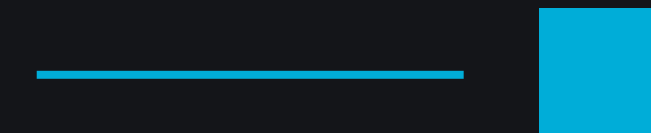
- Copia el valor
- No modifica el valor original

Receiver por puntero:

- Permite modificar el original
- Evita copiar structs grandes
- Necesario para algunos casos

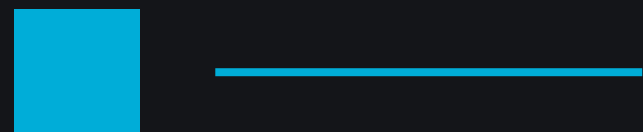


Tampa común: Mutabilidad y slices/maps

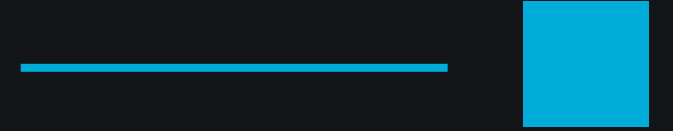


Aunque pases algún struct con valor: Un slice y un map adentro apuntan a datos compartidos.

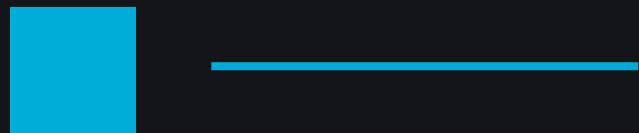
Pueden mutar el contenido sin usar puntero.



{dev/talles}



Interfaz en Go



¿Qué es una interfaz en Go?

Es un contrato de métodos (es un conjunto de métodos).

Se implementan implícitamente, no existe implements. Go no obliga a declarar 'implements X'. Si tienes los métodos, ya estás dentro

```
methods.go  
  
type Writer interface {  
    Write(p []byte) (n int, err error)  
}
```



¿Para qué sirven las interfaces?

- **Desacoplar** (Depender del comportamiento en vez de la implementación)
- Fácil para testing
- Permitir múltiples implementaciones.



Ventajas y desventajas de las interfaces

VENTAJAS:

- Código flexible
- Más fácil de probar
- Menos acoplamiento

DESVENTAJAS:

- Si las crear “por adelantado”, se complica todo.
- Interfaces grandes = dolor
- Abusar de interfaces puede esconder estructura real



Interfaces pequeñas: composición

Es mejor interfaces mínimas y las compones según la necesidad



methods.go

```
type Reader interface { Read(...) }
```

```
type Writer interface { Write(...) }
```

```
type ReadWriter interface { Reader; Writer }
```



La trampa del nil en interfaces

La interfaz en Go tiene: tipo dinámico y valor

```
tipo = *myErr  
valor = nil
```

Sin embargo la interfaz no es nil



¿Cuándo usar cada cosa?

Tipos propios: Cuando un valor tiene significado de negocio: Money, UserID, Email

Métodos: Cuando el comportamiento pertenece al tipo

Interfaces: Cuando quieres desacoplar y probar fácil.



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

